



北京航空航天大学
BEIHANG UNIVERSITY

操作系统Lab0课上测试简介



Lab0测试说明



- **考试时间 14:00 ~ 16:00**
- 每次课上测试题目分为**基础测试**和**附加测试**(选做)两部分
- 每题单独评分，满分都是100分
- **请注意，Lab得分为：Lab基础分值 * (课下成绩 * 0.6 + 课上exam成绩 * 0.4) / 100。**
- **附加测试加分为：通过 (>=60分) 课上测试Extra题目所给予的加分。**
- 这里需要用到的所有的指令**都在指导书中出现过**，你可以登录课程MOOC网站<https://os.buaa.edu.cn>下载指导书进行参考。



Step1:创建lab0-exam分支

- `cd ~/学号/`
- `git checkout lab0`
- `git checkout -b lab0-exam` (注意: 有参数-b)



Step2:完成lab0基础题代码编写



小提示

忘记了某些指令的用法？

- 可以使用man xxx查询手册。例如man find，可以查看find命令的手册
- 在使用man查询时，善用 /（斜杠）+ 回车搜索功能。例如可以使用 /filename回车查找和filename相关的内容
- 使用方向键（上下左右）导航
- 使用n查看下一个匹配内容，使用N（大写）查看上一个匹配内容
- **当然也可以看指导书**



Lab0课上基础题



任务1:

要求：在工作目录（如20xxxxxx）下编写Makefile完成以下功能：

1. 执行make命令后，可以将同目录下的hello_os.c编译成名为os_hello的程序（同学们仅需要完成Makefile的编写；若想要在本本地测试，可以自行编写hello_os.c。**注意：编译后的可执行文件与源程序文件名不一样**）。（25分）
2. 用make clean命令后，可以将编译出来的os_hello清除。（25分）



任务2:

要求：在工作目录（如20xxxxxx）下创建lab0-exam.sh完成以下功能：

1. `#!/bin/bash`后的第一行为make指令，调用前文所写的Makefile创建os_hello这个程序；
2. 在lab0-exam.sh文件同目录下创建一个hello_os的空文件；
3. 在lab0-exam.sh文件同目录下创建一个名为hello_os_dir的目录；
4. 将第一步的make命令创建的os_hello**复制**到hello_os_dir文件夹内；
5. 再复制一份os_hello文件到hello_os_dir文件夹内，并将其命名为hello_os（此时hello_os_dir中应有os_hello和hello_os两个文件）；
6. 清除和lab0-exam.sh文件同目录下的os_hello（即第一步make出来的os_hello）

(能够正确完成以上六个步骤得25分)



任务2（续上一页）：

7. 最后往hello_os.txt中输出所有包含hello_os.c中包含“os_hello”的行与行号，且不区分大小写，如第五行有“int os_HELLO=5”，则你要输出“5:int os_HELLO=5”。注意输出的行号的冒号后面就是这一行原本的内容，你不需要做任何更改。如果9 10 11 行如左图，则要输出为右图。（25分）

```
long OS_HELLO;  
OS_HELLO = 3;  
OS_HELLO = os_hello;
```

```
9:long OS_HELLO;  
10:OS_HELLO = 3;  
11:OS_HELLO = os_hello;
```



如果你正确的进行了lab0-exam.sh脚本的编写，运行后的文件树会如右图所示。

注2：只需要提交lab0-exam.sh，和Makefile，不要提交hello_os.c和其他文件。但是csc，src等课下任务的文件夹可以不删除，不会对评测进行影响，如果你没有删除，则右侧文件树还会有四个课下的文件夹。

```
.  
|-- Makefile  
|-- hello_os  
|-- hello_os.c  
|-- hello_os.txt  
|-- hello_os_dir  
|   |-- hello_os  
|   |-- os_hello  
|-- lab0-exam.sh
```



提交评测机文件树样例

建议同学们依照该文件树，
在本地建立一个类似的目录，
便于测试自己脚本正确性。

```
| - lab0-exam.sh  
| - Makefile
```

注2：只需要提交lab0-exam.sh，和Makefile，不要提交hello_os.c和其他文件。但是csc，src等课下任务的文件夹可以不删除，不会对评测进行影响，如果你没有删除，则右侧文件树还会有四个课下的文件夹。



评测逻辑：

1. 首先评测任务1第1问，通过得25分，否则终止评测，返回成绩；
2. 其次评测任务1第2问，通过得25分，否则终止评测，返回成绩；
3. 接着评测任务2第1-6问，通过得25分，否则终止评测，返回成绩；
4. 最后评测任务2第7问，通过得25分，否则终止评测，返回成绩；
5. 返回最终成绩。



Step3:提交评测

- `cd ~/学号/`
- `git add lab0-exam.sh Makefile`
- `git commit -a -m "balabala..."` (请将balabala改为有意义的信息)
- `git push origin lab0-exam:lab0-exam`



```
remote: *****
remote:
remote:          BUAA OSLAB AUTOTEST SYSTEM
remote:          Copyright (c) BUAA 2015-2022
remote:
remote: *****
remote:
remote: [ You are changing the branch: lab0-exam ]
remote:
remote: Branch 'lab0-exam' set up to track remote branch 'lab0-exam' from 'origin'.
remote: Switched to a new branch 'lab0-exam'
remote: Autotest: Begin at Wed Mar  2 00:48:06 CST 2022
remote:
remote:
remote: [ You have passed make test ]
remote:
remote: [ You have passed make clean test ]
remote:
remote: [ You have passed file test ]
remote: [ You have passed grep test ]
remote: [ You got 100 (of 100) this time. Wed Mar  2 00:48:09 CST 2022 ]
```



Lab0课上附加题 (选做)

**首先确保成功创建并切换到lab0-Extra分支
(注意分枝名中大小写严格按照上面的要求。
操作命令参见之前的Lab0测试说明。)**



Step5:创建lab0-Extra分支 (选做)

- 通过基础题后，可创建附加题分支
- `cd ~/学号/`
- `git checkout -b lab0-Extra`



Step6:完成lab0-Extra代码编写 (选做)



这里如果有**你觉得没有看到过的指令**，可以**在指导书中搜索关键字进行查询**，这个小技巧在后面lab的测试中没准也会用到。



课上测试附加题

题目背景：

我们写代码通过GCC进行编译的时候，有时会出现一些warning，甚至会出现error，比如右侧所提供的代码，就会出现一些warning，如下图，我们后面的讲述会以它为例。

```
#include<stdio.h>
int main()
{
    int a,b;
    if(2<3<4)
    {
        scanf("%d",&a);
        printf("%d\n",a+a);
    }
    return 0;
}
```

```
test.c: In function 'main':
test.c:5:8: warning: comparisons like 'X<=Y<=Z' do not have their mathematical meaning [-Wparentheses]
   5 |     if(2<3<4)
     |         ^
test.c:4:8: warning: unused variable 'b' [-Wunused-variable]
   4 |     int a,b;
     |         ^
```



任务：

本任务要求你在工作目录（即~/学号/）下写一个名为**find_warnings.sh**的bash 脚本文件，其用法为：**`./find_warnings.sh path n`**

第一个参数path：代表你要编译的C文件的绝对路径，评测机评测时会保证该C文件存在，如 “/root/test/hello.c”，这个C文件中的**文本内容不包括error和warning**；

第二个参数 n：一个整数，代表后面会用到的输入的上限，如 5。



任务1:

find_warnings.sh运行时，首先需要将参数path所表示的C文件编译为可执行文件test，并添加-Wall编译选项，以便输出更多错误和告警信息。将错误和警告信息存储到**warning.txt**文件中。

提示：将test.c 的错误和警告信息输出到a.txt的方式如下

```
gcc -Wall test.c -o test 2> a.txt
```

注意：“2>”为一个整体，之间不能有空格



任务2:

从前一步所生成的warning.txt中筛选出所有包含字符串“warning”的行，并去掉其中所有“warning: ”字样（去掉的是引号内的内容，注意引号内有一个空格），结果输出到result.txt中。



任务3:

如果参数path所指代路径的C文件**可以成功编译为可执行文件test**（即没有error），则将test程序运行n次（n为find_warning.sh运行时的参数）。第i次（从1开始计数）以正整数i做为test**从标准输入中读入的数据**，得到输出的结果，将这个输出结果**追加**到result.txt中。**注意，“追加”指的是“不能覆盖掉任务2和任务3运行的输出”。**

如果不能成功编译可执行文件test，则跳过本步骤。

提示：上一条命令执行的结果（编译是否成功）可以用\$?表示，
为0表示成功执行，非0为执行出错。
注意脚本中if和while的使用



任务4:

无论path所指代的C文件编译是否成功，都要**用指令**在result.txt中**追加输出**运行find_warnings.sh这个文件的**绝对路径**。

不记得使用什么命令的同学可以
试着在指导书中搜索关键词

*这里你可能会感到奇怪，**为什么输出的不是find_warnings.sh的路径，而输出的是运行它的路径，（参见后面的样例解释），甚至有可能不知道这里要用什么指令。***

由于这个指令非常关键，我们在这个PPT的最后一页直接告诉了你加粗的两个问题的答案，但是建议你在充分思考后进行阅读。



使用示例：

假设/home/git/test.c如右所示。

当你完成find_warnings.sh的编写后

假设你当前在/home/git目录下，

输入

bash ./学号/find_warnings.sh /home/git/test.c 5

```
#include<stdio.h>
int main()
{
    int a,b;
    if(2<3<4)
    {
        scanf("%d",&a);
        printf("%d\n",a+a);
    }
    return 0;
}
```



首先会在warning.txt输出

```
/home/git/test.c: In function 'main':  
/home/git/test.c:5:8: warning: comparisons like 'X<=Y<=Z' do not have their mathematical meaning [-Wparentheses]  
  5 |     if(2<3<4)  
    |           ^  
/home/git/test.c:4:8: warning: unused variable 'b' [-Wunused-variable]  
  4 |     int a,b;  
    |           ^
```

接着可在result.txt中得到

```
/home/git/test.c:5:8: comparisons like 'X<=Y<=Z' do not have their mathematical meaning [-Wparentheses]  
/home/git/test.c:4:8: unused variable 'b' [-Wunused-variable]  
2  
4  
6  
8  
10  
/home/git
```



```
/home/git/test.c:5:8: comparisons like 'X<=Y<=Z' do not have their mathematical meaning [-Wparentheses]
/home/git/test.c:4:8: unused variable 'b' [-Wunused-variable]
2
4
6
8
10
/home/git
```

说明：

- (1) result.txt的第1、2行，是从warning.txt中筛选出的包含字符串“**warning**”的行（即warning.txt的第2、5行）删去“warning:”的结果；
- (2) result.txt的第3~7行，为将1到5（脚本的第2个参数n为5）**分别**输入给可执行文件**test**得到的**多个输出**（由test.c的内容可知，把1输入给test则输出2，把2输入则输出4，以此类推）；
- (3) result.txt的第8行为运行find_warnings.sh时路径的**绝对路径**



10个测试数据，一个10分；

10%的数据不能正常编译且没有warning

30%的数据不能正常编译但有warning

30%的数据能正常编译但没有warning

剩下30%数据能正常编译且有warning



Step7:提交评测（选做）

- 必须通过基础测试才能获得附加题分数
- `cd ~/学号/`
- `git add find_warnings.sh`
- `git commit -a -m "balabala..."`（请将balabala改为有意义的信息）
- `git push origin lab0-Extra:lab0-Extra`



Step8:提交结果（选做）

```
remote: *****
remote:
remote:          BUAA OSLAB AUTOTEST SYSTEM
remote:          Copyright (c) BUAA 2015-2022
remote:
remote: *****
remote:
remote: [ You are changing the branch: lab0-Extra ]
remote:
remote: Switched to a new branch 'lab0-Extra'
remote: Branch 'lab0-Extra' set up to track remote branch 'lab0-Extra' from 'origin'.
remote: Autotest: Begin at Wed Mar  2 00:52:00 CST 2022
remote:
remote: [ You have passed testcase 1/10. ]
remote: [ You have passed testcase 2/10. ]
remote: [ You have passed testcase 3/10. ]
remote: [ You have passed testcase 4/10. ]
remote: [ You have passed testcase 5/10. ]
remote: [ You have passed testcase 6/10. ]
remote: [ You have passed testcase 7/10. ]
remote: [ You have passed testcase 8/10. ]
remote: [ You have passed testcase 9/10. ]
remote: [ You have passed testcase 10/10. ]
remote: [ You got 100 (of 100) this time. Wed Mar  2 00:52:15 CST 2022 ]
```



北京航空航天大学
BEIHANG UNIVERSITY

下面请同学们开始做题
有问题可以随时提问

完成之后及时提交，请勿卡点提交，避免网络拥堵导致无法及时评测



关于Extra的步骤4的附加说明

完成步骤4你只需要使用**pwd**指令即可，**只需要这三个字母即可完成步骤四**。你可以通过在指导书中搜索“绝对路径”即可找到这个命令，不会在pdf搜索关键词的同学可以学习一下😊

关于为什么输出的不是find_warnings.sh的路径，而输出的是运行它的路径：

你可以大致理解为Shell脚本只是把命令写到了一个文件中，而运行这个Shell脚本只是在运行这个Shell脚本的目录运行Shell脚本中所写的命令，和Shell脚本所在目录无关。

你可以做一个有趣的测试：在你的工作目录下创建一个文件夹叫test，并创建一个Shell脚本名为test_bash.sh，除#!/bin/bash后的第一行为cd test，第二行为pwd。分别在你的工作目录/home/git/学号 输入bash test_bash.sh运行这个脚本，和/home/git 输入 bash ./学号/test_bash.sh来运行这个脚本，看看结果有什么不同。如果你以后写Bash脚本一定要小心这个问题，不然如果里面有rm命令，然后在过高层调用了这个脚本，结果可想而知😞